
Assignment Name: Project 2: Optimizing GEMM for Deep-Learning

Class Name: ECE452

Date Submitted: 4/12/2026

Group Members: Raghav Bharath (rsb49)

Introduction:

For this project, I explored how hardware-aware optimizations affect the performance of General Matrix Multiplication (GEMM), the core computational kernel behind deep learning operations like convolutions in CNNs. I started by measuring the CPU cache hierarchy to understand the memory constraints for tiling, then implemented a naive GEMM using pure Python triple nested loops and benchmarked it against NumPy to separate interpreter overhead from actual computation time. From there, I extended the implementation to a cache-aware tiled GEMM using 6 nested loops, testing block sizes of 16, 32, 64, and 128 across matrix sizes of 128x128, 256x256, and 512x512. Finally, I ran the same operation on an NVIDIA Tesla T4 GPU via Google Colab using PyTorch as an interface to NVIDIA's cuBLAS library and compared all results to quantify the speedup achievable through GPU parallelism.

Part 1

For Part 1, I determined my CPU's cache hierarchy by running the following command in the WSL terminal:

```
- lscpu | grep -i cache
```

This gave me the following results:

- L1 Cache: 49,152B (48 KB)
- L2 Cache: 1,310,720B (1.25 MB)
- L3 Cache: 25,165,824B (24MB)

The cache hierarchy is the layers of memory that sit between the CPU and the main RAM. These results basically told me what the size constraints are when we choose a tile size for GEMM. A 64x64 FP32 tile occupies $64 \times 64 \times 4 = 16,384$ bytes (16 KB), which fits comfortably in the 48 KB L1 cache. A 128x128 tile occupies 64 KB, which will exceed the L1 cache and spill into the L2 cache. This explains why, later on in Part 3, a block size of 64 outperformed the block size of 128.

Part 2

For Part 2, I implemented a naive GEMM approach using a triple nested loop in pure Python, following $C = A \times B + C$. Because pure Python creates a lot of interpreter overhead, I also ran the same operation using NumPy's `@` operator to compare against an optimized baseline. So, I generated the matrices using `np.random.randn(size, size).astype(np.float32)` and tested at sizes 128x128, 256x256, and 512x512. You can see the average results below (I've also attached the detailed results in the zip folder).

Size	Avg Python (seconds)	Avg NumPy (seconds)	Speedup
128x128	1.2225	0.000913	1339.4x
256x256	9.2633	0.027003	343.0x
512x512	75.9566	0.030352	2502.5x

The massive gap between Python and NumPy comes down to how each of them handles computation. NumPy works with a lower level of the memory hierarchy and allows for cache prefetching with sequential data, which provides faster access. Python's interpreter actually carries around 28 bytes of overhead per integer object (I tested this by importing `sys` and running `sys.getsizeof(1)`), which means that each MAC operation involves memory allocation and reference counting on top of the actual math. However, when we use NumPy's `@` operator, it actually dispatches to an optimized C/Fortran backend that uses raw contiguous FP32 memory and SIMD vectorization.

How does execution time change as the matrix size increases?

The execution time scales at $O(n^3)$ because of the triple-nested loop. This means that doubling the matrix size would equal multiplying the runtime by 8x. With the naive Python implementation, we went from 1.22s at 128x128 to 75.96s at 512x512.

- $512/128 = 4$
- $4^3 = 64$ (This is how much the execution time should scale by)
- $1.22 \times 64 = 78.08 \approx 75.96$ s, so the results work out to be relatively accurate.

This is also expected because GEMM requires n^3 MAC operations for an $n \times n$ matrix. Something I noticed, however, was that although NumPy should scale cubically in theory, the actual times don't scale as cleanly and in the sub-millisecond range. For example, 256x256 at 0.027s is around 30x slower than 128x128, and 512x512 at 0.030s is barely even slower than 256x256. Based on online research, I think this is possibly due to the system-level noise at that timescale. It could be OS scheduling or even memory allocation variance, so it doesn't have to be anything meaningful about the algorithm. Also, I only averaged 3 iterations, which may not have been enough to smooth out that kind of variance for operations that are this fast. Therefore, the Python scaling is more reliable than the $O(n^3)$ scaling here.

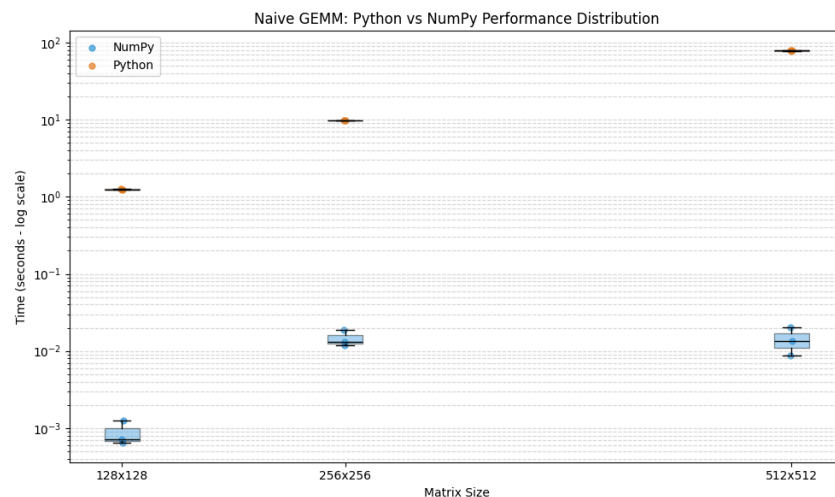


Figure 1. Python vs NumPy Performance Distribution

Part 3

For Part 3, I modified the naive GEMM to use a cache-aware tiled implementation. Instead of iterating over entire rows and columns, the computation was broken into small tiles using 6 nested loops. There were 3 outer loops (f, g, h) to select tile positions, and 3 inner loops (i, j, k) to compute within each tile. This keeps the working data set resident in L1 cache for the duration of each tile rather than repeatedly fetching new data from slower memory. Then, I tested block sizes of 16, 32, 64, and 128 across all three matrix sizes, and ran both the Python and NumPy versions of the tiled implementation.

Python Tiled Results:

Block	128x128 (seconds)	256x256 (seconds)	512x512 (seconds)
16	1.2762	9.9473	80.0421
32	1.2703	9.0732	77.8905
64	1.1884	9.4734	75.7319
128	1.2101	9.3861	77.2036

NumPy Tiled Results:

Block	128x128 (seconds)	256x256 (seconds)	512x512 (seconds)
16	0.004827	0.026627	0.195474
32	0.000736	0.004878	0.043756
64	0.000353	0.001928	0.015951
128	0.000994	0.009544	0.021241

Which block size provides the best performance?

The block size of 64 provided the best performance across both implementations. For Python, the differences are marginal since interpreter overhead dominates regardless of tile size. The NumPy results are a bit clearer, though. Block=64 at 0.01595s for 512x512 was the fastest, significantly beating block=16 at 0.1955s and block=32 at 0.04376s. This makes sense given the cache sizes from Part 1: a 64x64 FP32 tile occupies $64 \times 64 \times 4 = 16,384$ bytes (16 KB), which fits comfortably within the 48 KB L1 cache. Block=32 tiles are only 4 KB, which fits in L1 but leaves most of it unused. Block=16 tiles are only 1 KB, so the overhead of managing more tiles ends up outweighing the cache benefit.

What happens when a block size exceeds the L1 or L2 cache size?

A 128x128 tile occupies $128 \times 128 \times 4 = 65,536$ bytes (64 KB), which exceeds the 48 KB L1 cache and forces data to spill into L2. This is visible in the NumPy results, where block=128 at 0.02124s was slower than block=64 at 0.01595s for 512x512. Once the tile no longer fits in L1, the CPU has to fetch data mid-tile from L2, which partially defeats the purpose of tiling. The sweet spot ends up being a tile size that fits fully in L1 but is still large enough to make the tiling worthwhile, which in this case is block=64.

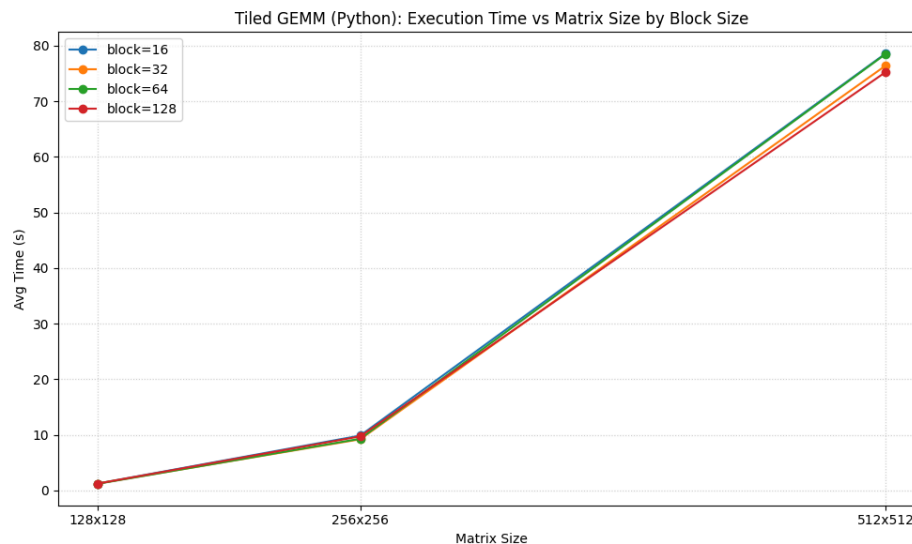


Figure 2. Python Execution Time vs Matrix Size

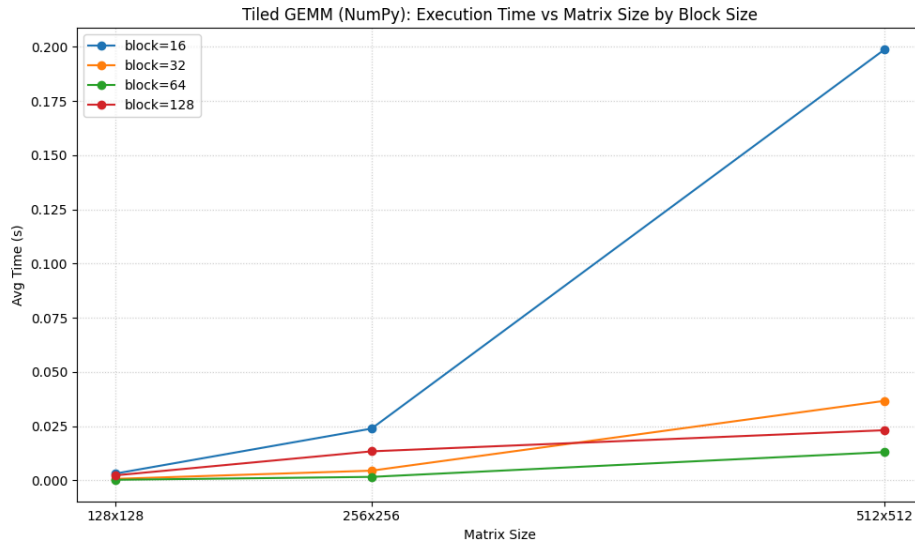


Figure 3. NumPy Execution Time vs Matrix Size

Parts 4 & 5 - GPU cuBLAS and Performance Comparison

For Part 4, I ran the same GEMM operation on an NVIDIA Tesla T4 GPU via Google Colab, using PyTorch as an interface to NVIDIA's cuBLAS library, since `torch.matmul` on a CUDA tensor automatically dispatches to cuBLAS under the hood. I allocated the matrices directly on the GPU using `device='cuda'` and timed it using CUDA events rather than a CPU timer. This is because the CPU queues the GPU work asynchronously and would return before the GPU actually finishes. I also used a warmup run before timing to exclude one-time CUDA kernel initialization overhead. When I did some research, I found that the first call to any CUDA operation triggers a one-time initialization where the GPU loads and compiles the cuBLAS kernels, allocates context, and sets up the CUDA runtime. This adds a lot of latency that has nothing to do with the actual GEMM computation, so I wanted to get it out of the way.

Note: I averaged the results over 10 iterations

Size	cuBLAS Avg (seconds)
128x128	0.000073
256x256	0.000078
512x512	0.000166

How does cache-aware GEMM compare to naive GEMM in execution speed?

Looking at all implementations together, the performance gap between each approach is massive. Python naive at 512×512 took 75.96s. NumPy tiled at `block=64` brought that down to 0.01595s, a roughly 4,760x improvement, by actually respecting cache locality through a compiled backend. cuBLAS on the T4 finished the same computation in 0.000166s, which is about 457,600x faster than Python naive and about 96x faster than the best CPU tiled result. The key takeaway is that tiling only meaningfully helps when the runtime actually respects memory layout. In Python loops, the interpreter overhead swamps any cache benefit. In NumPy and cuBLAS, tiling and parallelism are handled at the hardware level, which is where the real gains come from.

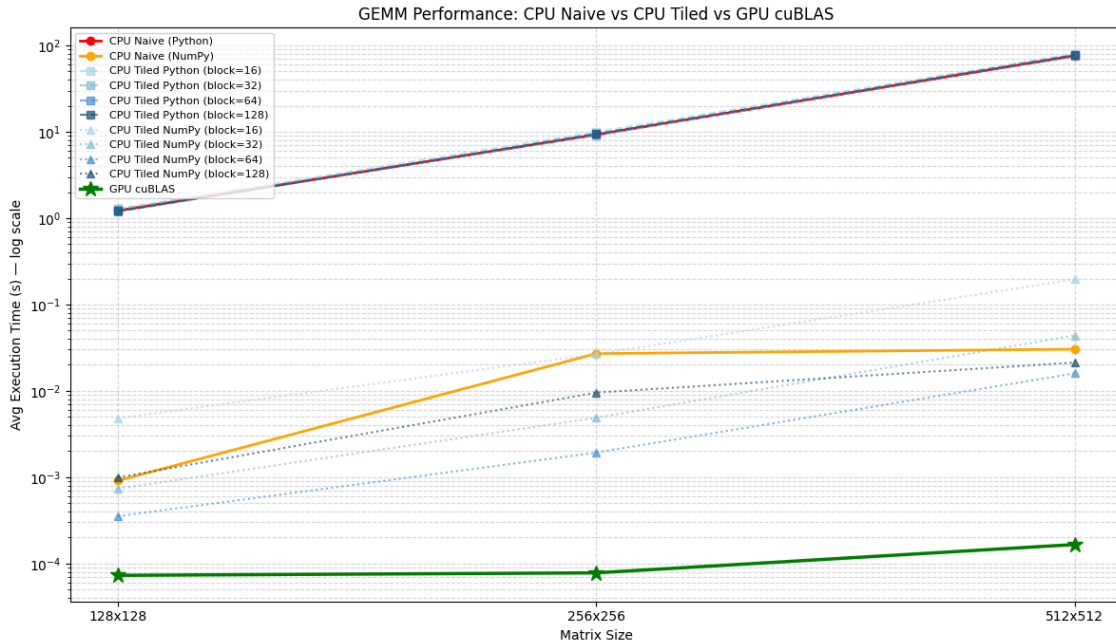


Figure 4. Performance Comparison - CPU Naive vs CPU Tiled vs GPU cuBLAS

Conclusion

This project showed that GEMM performance is determined more by how well an implementation maps to hardware than by the algorithm itself. By measuring the cache hierarchy upfront, I was able to predict that block=64 would be optimal before even running the tiled experiments, since a 64x64 FP32 tile fits cleanly within the 48 KB L1 cache. The Python results confirmed that tiling only helps when the runtime actually respects memory layout, since interpreter overhead completely swamps any cache benefit in pure loops. Moving to NumPy's compiled backend made the cache-aware advantage visible, and moving to GPU cuBLAS made all CPU optimizations look minor by comparison. I came away from this project with a much more concrete understanding of how cache hierarchy, memory layout, and hardware parallelism interact to determine real-world performance for data-parallel workloads like GEMM.

References

- [1]. NVIDIA Corporation, "cuBLAS Library," docs.nvidia.com, 2025, <https://docs.nvidia.com/cuda/cublas/>
- [2]. PyTorch Contributors, "torch.matmul," pytorch.org, 2025, <https://pytorch.org/docs/stable/generated/torch.matmul.html>
- [3]. NumPy Contributors, "numpy.matmul," numpy.org, 2025, <https://numpy.org/doc/stable/reference/generated/numpy.matmul.html>